

Experiment No: 1

Title: Cryptography in Blockchain, Merkle Root Tree Hash

Aim

To study and implement cryptographic concepts used in Blockchain technology including SHA-256 hashing, Proof-of-Work cryptographic puzzles, Golden Nonce generation, and Merkle Tree Root Hash construction.

Objectives

1. To understand the role of cryptographic hash functions in Blockchain.
2. To generate cryptographic hashes using SHA-256 algorithm.
3. To understand the concept of nonce and Proof-of-Work mechanism.
4. To implement a program for finding the Golden Nonce.
5. To construct a Merkle Tree and generate the Merkle Root Hash.
6. To understand how Merkle Trees provide transaction integrity in Blockchain.

Theory

1. Cryptographic Hash Functions in Blockchain

A cryptographic hash function is a mathematical function that converts input data of any size into a fixed-size output called a hash value.

Blockchain uses cryptographic hash functions to maintain security, integrity, and immutability of data.

The SHA-256 algorithm is commonly used in Bitcoin blockchain to generate secure hash values.

Properties of Cryptographic Hash Functions

1. **Deterministic:**

The same input always produces the same hash value.

2. **One-way Function:**

The original input data cannot be easily obtained from the generated hash.

3. Collision Resistance:

It is extremely difficult for two different inputs to generate the same hash value.

4. Avalanche Effect:

A small change in input produces a completely different hash output.

2. Cryptographic Puzzle and Golden Nonce

A cryptographic puzzle is a computational problem used in blockchain mining.

In Proof-of-Work, miners repeatedly change the nonce value and calculate the hash until they find a valid hash that satisfies the required difficulty condition.

The nonce value that generates the required hash is called the Golden Nonce.

The Golden Nonce is used by miners to successfully add a new block to the blockchain.

3. Proof-of-Work

Proof-of-Work is a consensus mechanism used by Bitcoin to validate transactions and add new blocks to the blockchain.

Miners compete to solve a cryptographic puzzle by finding a nonce value that produces a hash with a specific number of leading zeros.

The miner who finds the valid nonce first gets the opportunity to add the block to the blockchain.

4. Merkle Tree

A Merkle Tree is a data structure used in blockchain to efficiently store and verify transactions.

In a Merkle Tree:

- Each transaction is converted into a hash value.
- Hash values are combined in pairs.
- The combined hashes are hashed again.
- This process continues until a single hash value remains.
- The final hash value is called the Merkle Root Hash.

The Merkle Root represents all transactions present in a block.

5. Working of Merkle Tree

The working process of a Merkle Tree is as follows:

1. Transactions are collected in a block.
2. Each transaction is converted into a cryptographic hash.
3. Transaction hashes are paired and combined.
4. New hashes are generated from combined hashes.
5. The process continues until the root hash is obtained.
6. The final hash is stored in the block header as the Merkle Root.

6. Benefits of Merkle Tree

- Provides efficient transaction verification.
- Reduces the amount of data required for verification.
- Detects any modification in transaction data.
- Improves blockchain scalability.
- Provides data integrity and security.

7. Applications of Merkle Tree

Merkle Trees are used in:

1. Bitcoin Blockchain
2. Ethereum Blockchain
3. Cryptocurrency wallets
4. Distributed storage systems
5. File verification systems

Tasks Performed

Task 1: SHA-256 Hash Generation

A Python program was implemented to generate SHA-256 cryptographic hash values for a given input string using the hashlib library.

The generated hash value provides a unique digital fingerprint of the input data.

Task 2: Target Hash Generation Using Nonce

A program was implemented to generate a hash value by combining input data with a nonce value.

The nonce value changes the hash output and is used to simulate the mining process in blockchain.

Task 3: Proof-of-Work Puzzle Solving

A Proof-of-Work program was implemented to find the Golden Nonce.

The program continuously changes the nonce value until a hash satisfying the required difficulty condition is generated.

Task 4: Merkle Tree Root Hash Construction

A Merkle Tree was constructed using multiple transactions.

The transactions were first converted into hash values.

The intermediate hashes were generated by combining transaction hashes.

The final hash generated represents the Merkle Root Hash.

Programs Performed

Program 1: SHA-256 Hash Generation

```
import hashlib

def create_hash(string):

    hash_object = hashlib.sha256()

    hash_object.update(string.encode('utf-8'))

    hash_string = hash_object.hexdigest()

    return hash_string
```

```
input_string = input("Enter a string: ")

hash_result = create_hash(input_string)

print("Hash:", hash_result)
```

Output:

Enter a string: Prakash

Hash: c4ed7db2b936a1e7bcbf33c01a3e3319eea221801242cd24a417af68e2a47456

Program 2: Hash Generation Using Nonce

```
import hashlib

input_string = input("Enter a string: ")

nonce = input("Enter the nonce: ")

hash_string = input_string + nonce

hash_object = hashlib.sha256(hash_string.encode('utf-8'))

hash_code = hash_object.hexdigest()

print("Hash Code:", hash_code)
```

Output:

Enter a string: Block Chain

Enter the nonce: 3

Hash Code:

f526d38980e4bbd89c0d1fcff526912a38a18daea2e3964058ac8233a2579763

Program 3: Proof-of-Work Puzzle and Golden Nonce

```
import hashlib

def find_nonce(input_string, num_zeros):

    nonce = 0

    hash_prefix = '0' * num_zeros

    while True:

        hash_string = input_string + str(nonce)

        hash_object = hashlib.sha256(hash_string.encode('utf-8'))

        hash_code = hash_object.hexdigest()

        if hash_code.startswith(hash_prefix):

            print("Hash:", hash_code)

            return nonce

        nonce += 1

input_string = "A"

num_zeros = 1

expected_nonce = find_nonce(input_string, num_zeros)

print("Input String:", input_string)

print("Leading Zeros:", num_zeros)
```

```
print("Expected Nonce:", expected_nonce)
```

Output:

Hash: 06663975f75f189f1d70bd14d8f22df264cd6a5c7575d6875183d0ec76432fbb

Input String: A

Leading Zeros: 1

Expected Nonce: 9

Program 4: Merkle Tree Root Hash Construction**Description:**

This program creates a Merkle Tree from a list of transactions.

The transactions used are:

1. Alice → Bob : \$200
2. Bob → Dave : \$500
3. Dave → Eve : \$100
4. Eve → Alice : \$300
5. Roo → Bob : \$50

The program performs the following steps:

- Hashes all transactions.
- Combines transaction hashes.
- Generates intermediate hashes.
- Produces the final Merkle Root Hash.

```
import hashlib
```

```
def hash_data(data):
```

```
    return hashlib.sha256(
```

```
        data.encode('utf-8')
```

```
    ).hexdigest()
```

```
def build_merkle_tree(transactions):  
  
    if len(transactions) == 0:  
  
        return None  
  
    # Hash transactions first  
  
    transactions = [  
  
        hash_data(tx)  
  
        for tx in transactions  
  
    ]  
  
    print("Transaction Hashes:")  
  
  
    for tx in transactions:  
  
        print(tx)  
  
  
    while len(transactions) > 1:  
  
        if len(transactions) % 2 != 0:  
  
            transactions.append(transactions[-1])  
  
  
        new_transactions = []  
  
        print("\nIntermediate Hashes:")
```

```

    for i in range(0, len(transactions), 2):

        combined = transactions[i] + transactions[i+1]

        hash_combined = hash_data(combined)

        print(hash_combined)

        new_transactions.append(hash_combined)

    transactions = new_transactions

    return transactions[0]

```

```

transactions = [

    "Alice -> Bob : $200",

    "Bob -> Dave : $500",

    "Dave -> Eve : $100",

    "Eve -> Alice : $300",

    "Roo -> Bob : $50"

]

```

```

merkle_root = build_merkle_tree(transactions)

print("\nMerkle Root:")

print(merkle_root)

```

Output:

Transaction Hashes:

f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760
768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46
43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890da
cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a
cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac

Intermediate Hashes:

471608d3d6f2a642ff3a84dfaeeaebb1c271a9aed5b37a82d61c4784558e80e9
a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7
314ca746b3dafa63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e

Intermediate Hashes:

ae847393c3dff60e1d95cfdbb1c1c886d0bf7cbd26410bcc2469e023e94cb8ee
6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45

Intermediate Hashes:

96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0

Merkle Root:

96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0

Conclusion

This experiment demonstrated the importance of cryptography in Blockchain technology.

SHA-256 hashing provides secure data integrity by generating unique hash values. Proof-of-Work demonstrates how miners solve cryptographic puzzles by finding valid nonce values. The Golden Nonce represents the solution that satisfies blockchain difficulty requirements.

The Merkle Tree implementation demonstrated how blockchain efficiently summarizes and verifies multiple transactions using a single Merkle Root Hash.

Thus, cryptographic hashing, Proof-of-Work, and Merkle Trees together provide security, immutability, and trust in decentralized blockchain systems.